

DEVELOPER PORTFOLIO

안도형

Backend Engineer —

EMAIL dksehgud1@naver.com

GITHUB github.com/dksehgud

PHONE 010-2613-3584

ALGORITHM 삼성전자 SW 역량테스트 B형 보유

"개발 단계에서 사용자 불편을 먼저 생각하는
백엔드 개발자"

- 생각하는 방식

앱이나 웹 서비스를 사용할 때도 이 기능이 뒤에서 어떻게 동작할지 자연스럽게 고민합니다.
- 사용자 상황

새로고침·이탈·재접속처럼 실제 사용 중 생길 수 있는 상황을 먼저 정리합니다.
- 단순한 구조

기능을 어렵게 감싸기보다, 문제 크기에 맞는 단순한 구조와 기준을 먼저 선택합니다.
- 검증 범위

측정한 조건과 제외 범위를 함께 남겨, 결과를 설명할 수 있는 수준으로 검증합니다.

Skills

Java

Spring Boot

Spring Security

JPA

PostgreSQL

MySQL

Redis

Docker

ArgoCD

NAME

안도형 (Ahn Do-hyung)

ROLE FOCUS

Backend

CORE STACK

Java · Spring Boot ·
PostgreSQL/MySQL · Redis ·
Docker

CERTIFICATION

삼성전자 SW 역량테스트 B형

AFFILIATION

가천대학교 졸업
SSAFY 14기 수료

EDUCATION

- 2025.07 ~ 2026.06
삼성청년SW·AI아카데미 (SSAFY)
SSAFY 14기 Java 과정 수료 및 팀 프로젝트 수행
- 2025.02
가천대학교 컴퓨터공학과 학사 졸업

CERTIFICATIONS & TEST

SQLD

SQL 개발자 · 2026.06.19

ADsP

데이터분석 준전문가 · 2025.11.28

네트워크관리사 2급

2022.11.22

삼성전자 SW 역량테스트 B형

자료구조와 알고리즘 구현 역량 검증

AWARDS

- 2026.05.21
자율 프로젝트 우수상 — ReBloom
- 2026.04.03
삼성전자 VD·네트워크사업부 연계 프로젝트 우수상
- 2026.02.09
공통 프로젝트 우수상 — RE:FIT

Back-End

API & Business Logic

Spring Boot 3.x

Spring Security

JPA

Spring Boot REST API·비즈니스 로직 구현,
Spring Security 인증·인가 적용, JPA 엔티티·조회
로직 작성

Infra & DevOps

Cloud & Pipeline

Docker

ArgoCD

Kubernetes

Dockerfile·Docker Compose 작성, Kubernetes
Deployment·Service·Ingress 구성, ArgoCD 배포
상태 관리

Data & Queue

RDB & In-Memory

PostgreSQL

MySQL

Redis

PostgreSQL·MySQL 테이블·조회 쿼리 작성, Redis
Sorted Set·TTL·Lua Script 기반 대기열 구현

PROJECTS LIST

총 4개의 프로젝트 상세 내역

01	ReBloom 우수상	AIoT 스마트 스피커 ‘블루밍’ 기반 청소년 심리 케어 서비스	Spring Boot · EKS · ArgoCD · PostgreSQL
02	RE:FIT 우수상	중고 의류 거래 서비스의 배포와 CI/CD를 구축한 프로젝트	Spring Boot · Docker · Blue/Green
03	Tget	Redis 대기열로 예매 진입 순서를 제어한 공연 예매 서비스	Spring Boot 3 · Redis · MyBatis · k6
04	S-Catch 우수상	삼성닷컴 상품 정보를 수집·비교한 크롤링 프로젝트	Python · Playwright · OpenAI API · 결과 검증

EKS 인프라 · 관계 API · 리포트 API

ReBloom

청소년의 이상 신호를 보호자에게 알리고, 즉시 대응이 어려울 때 AIoT 스마트 스피커 ‘블루밍’이 대화를 시작해 분석 결과를 상담에 활용하는 심리케어 서비스.

보호자·자녀·상담사 관계 및 리포트 API를 개발하고, MSA의 7개 서비스를 EKS·ArgoCD 환경에 배포했습니다.

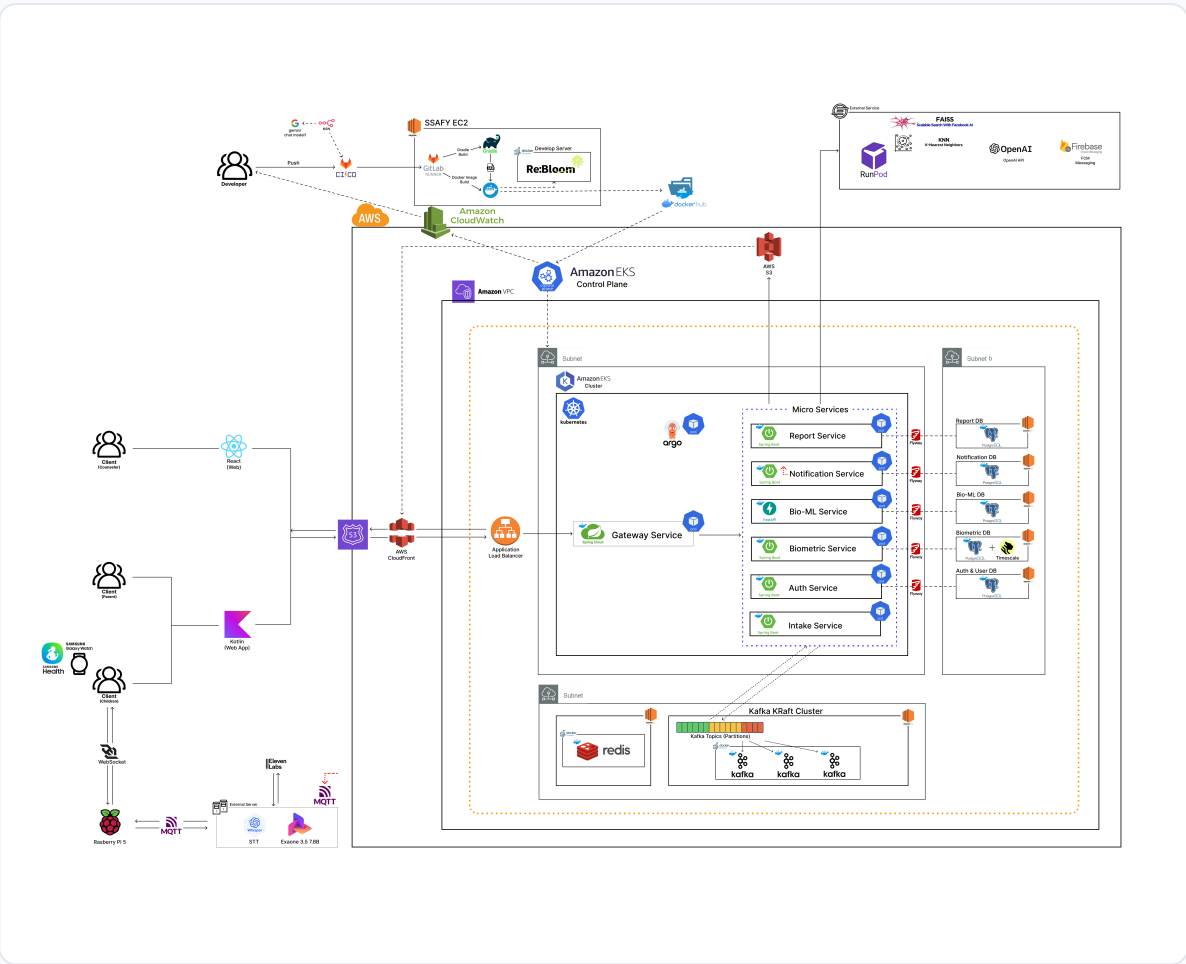
기간 2026.04 ~ 2026.05 (6주)

팀 구성 6명 (BE 2, FE 2, AI 2)

GitHub github.com/dksehgud/ReBloom

시스템 아키텍처

System Architecture



7개 서비스의 Kubernetes 리소스와 ALB Ingress·HTTPS, ArgoCD를 구성하고 report-service에 PostgreSQL JDBC 기반 ShedLock을 적용했습니다.

- AWS EKS
- Kubernetes
- ArgoCD
- PostgreSQL
- ShedLock
- ALB · HTTPS

프로젝트 종료 후 후속 검증

01

발견·재현

후속 검증에서 3개 Pod 중 1개가 Ready 상태로 전환되지 않았습니다.

로그에서 SQLState 53300을 확인했습니다. 배포 설정을 계산하니 max_connections=40인 검증용 RDS에서 Pod당 최대 20개, 총 60개 연결을 열 수 있는 구조였습니다.

원인·영향

Pod별 커넥션 풀이 합산되어 설정상 최대 60개가 되었고, DB 한도를 넘어 새 Pod가 기동하지 못했습니다.

선택 이유·적용

DB 사양을 높이기보다 서비스 규모에 맞춰 연결 예산부터 조정했습니다. 풀을 20개에서 8개로 낮추고 기존 Pod를 축소해 연결을 반환한 뒤 재배포했습니다.

검증 결과

60 → 24

설정상 최대 연결 수
실제 JDBC 세션 24개
Ready 2/3 → 3/3

02

발견·재현

자녀별 리포트의 날짜 범위 조회가 데이터 증가 시 전체 테이블을 훑었습니다.

365,000건의 합성 데이터를 적재하고 EXPLAIN ANALYZE를 실행해 Parallel Seq Scan과 DB 쿼리 p95 41.292ms를 확인했습니다.

원인·영향

children_id와 report_date를 함께 사용하는 조회 조건을 직접 지원하는 인덱스가 없었습니다.

선택 이유·적용

자녀 ID의 동등 조건과 날짜 범위 조건에 맞춰 복합 인덱스를 설계하고, 동일 스키마를 재현할 수 있도록 Flyway로 반영했습니다.

검증 결과

41.292 → 0.309ms

DB 쿼리 p95 99.3% 감소
Bitmap Index Scan 전환

추가 개선 코드 점검 중 보호자 목록 반복문 안에서 활성 자녀 관계를 다시 조회하는 1+N 구조를 발견해, 보호자 ID IN 조회와 Map 결합으로 쿼리 호출 구조를 1+N회에서 2회로 변경했습니다.
검증 범위 연결 수는 AWS EKS·RDS·CloudWatch, 조회 시간은 Docker PostgreSQL 환경의 DB 쿼리를 기준으로 확인했습니다.

RE:FIT

AI 가상 피팅과 중고 의류 거래 기능을 제공하는 플랫폼.

인프라·CI/CD를 담당해 Backend·Frontend·AI 실행 환경을 Docker로 구성하고, GitLab CI/CD와 Nginx Blue/Green 기반 배포 자동화를 구현했습니다.

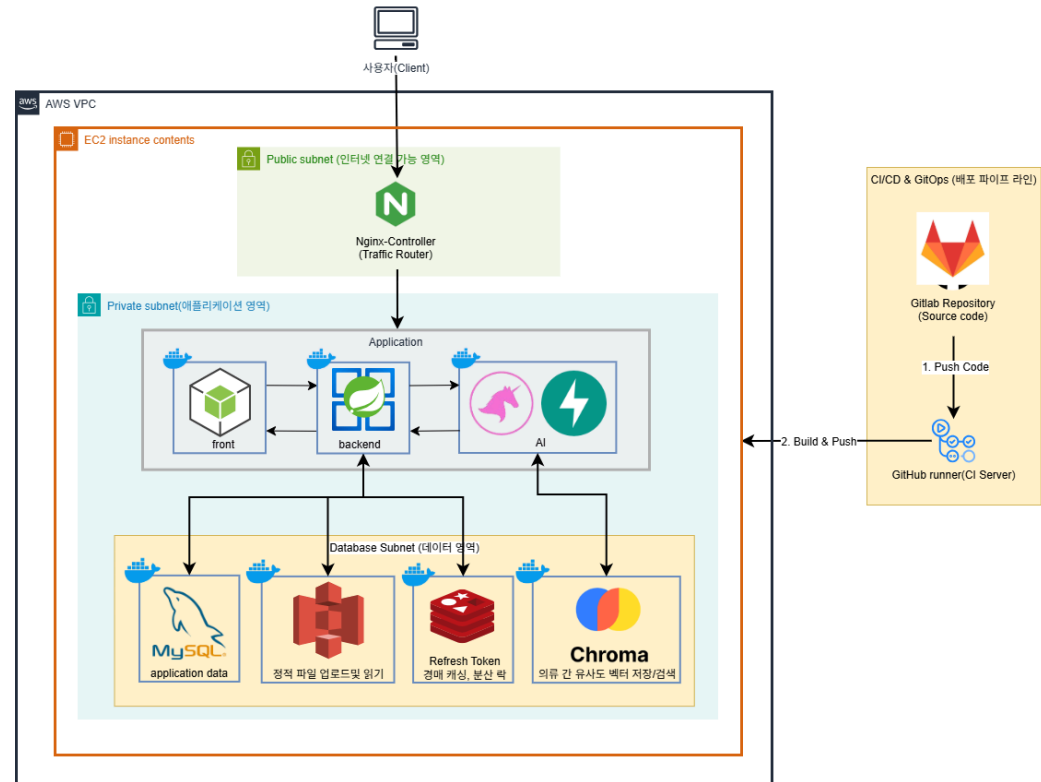
기간 2026.01 ~ 2026.02 (6주)

팀 구성 6명

GitHub github.com/dksehgud/ReFit

시스템 아키텍처

System Architecture



Backend·Frontend·AI 실행 환경을 Docker로 통일하고, GitLab CI/CD와 Nginx Blue/Green 구조로 빌드 검증부터 배포 전환까지 자동화했습니다.

Docker

Nginx

GitLab CI/CD

Docker Compose

Redis

MySQL

구축 과정의 설계 판단과 측정 결과

01

발견·측정

MR 빌드 로그에서 코드 변경마다 같은 Gradle 의존성을 다시 처리하는 구간을 확인했습니다.

동일 조건 재빌드에 37.4초가 걸렸고, 모노레포의 변경 영역과 무관한 작업까지 실행되어 피드백이 늦어졌습니다.

원인·영향

Gradle 의존성 캐시가 재사용되지 않았고
Backend·Frontend·AI의 변경 범위를 구분하지 않았습
니다.

선택 이유·적용

소스보다 변경 빈도가 낮은 Gradle 의존성은 캐시로 재사용하고,
rules:changes로 변경된 영역의 Job만 실행하도록 분리했습니다.

검증 결과

37.4 → 3.5초

동일 조건 Gradle 재빌드
90.6% 단축

02

요구사항·위험

기존 컨테이너를 먼저 종료하면 새 버전이 준비될 때까지 요청 처리 대상이 사라질 위험이 있었습니다.

단일 서버의 Docker·Nginx 구조를 유지하면서 배포 시점의 트래픽 전환 범위를 작게 만드는 방법이 필요했습니다.

원인·영향

기존 컨테이너를 먼저 종료하면 새 컨테이너의 기동 시간 동
안 요청을 처리할 대상이 사라집니다.

선택 이유·적용

기존 Nginx를 그대로 활용할 수 있는 Blue/Green 방식을 선택해 새 컨
테이너를 먼저 기동하고 upstream 전환 후 이전 컨테이너를 종료하도
록 자동화했습니다.

구현 범위

트래픽 전환 자동화

새 컨테이너 기동
Nginx upstream 전환
이전 컨테이너 종료

현재 공개 파이프라인 변경 영역별 build-check와 deploy를 실행합니다. Flyway migration·MySQL/Redis smoke·JPA integration Job은 구성·시험 후 최종 설정에서는 비활성화했습니다.
검증 한계 배포 중단 시간과 자동 롤백은 별도로 측정하지 않았습니다.

Redis 순번 · 원자 처리 · 이탈 회수 · 부하 테스트

Tget

공연 예매 오픈 시 물리는 요청을 대기열로 제어해 사용자를 순서대로 입장시키는 서비스.

백엔드 전반과 대기열을 담당했습니다. 프로젝트 종료 후 Redis 접수 순번과 원자 처리 방식을 적용한 개선안을 만들고, 순서·상태 변경·이탈 회수를 다시 검증했습니다.

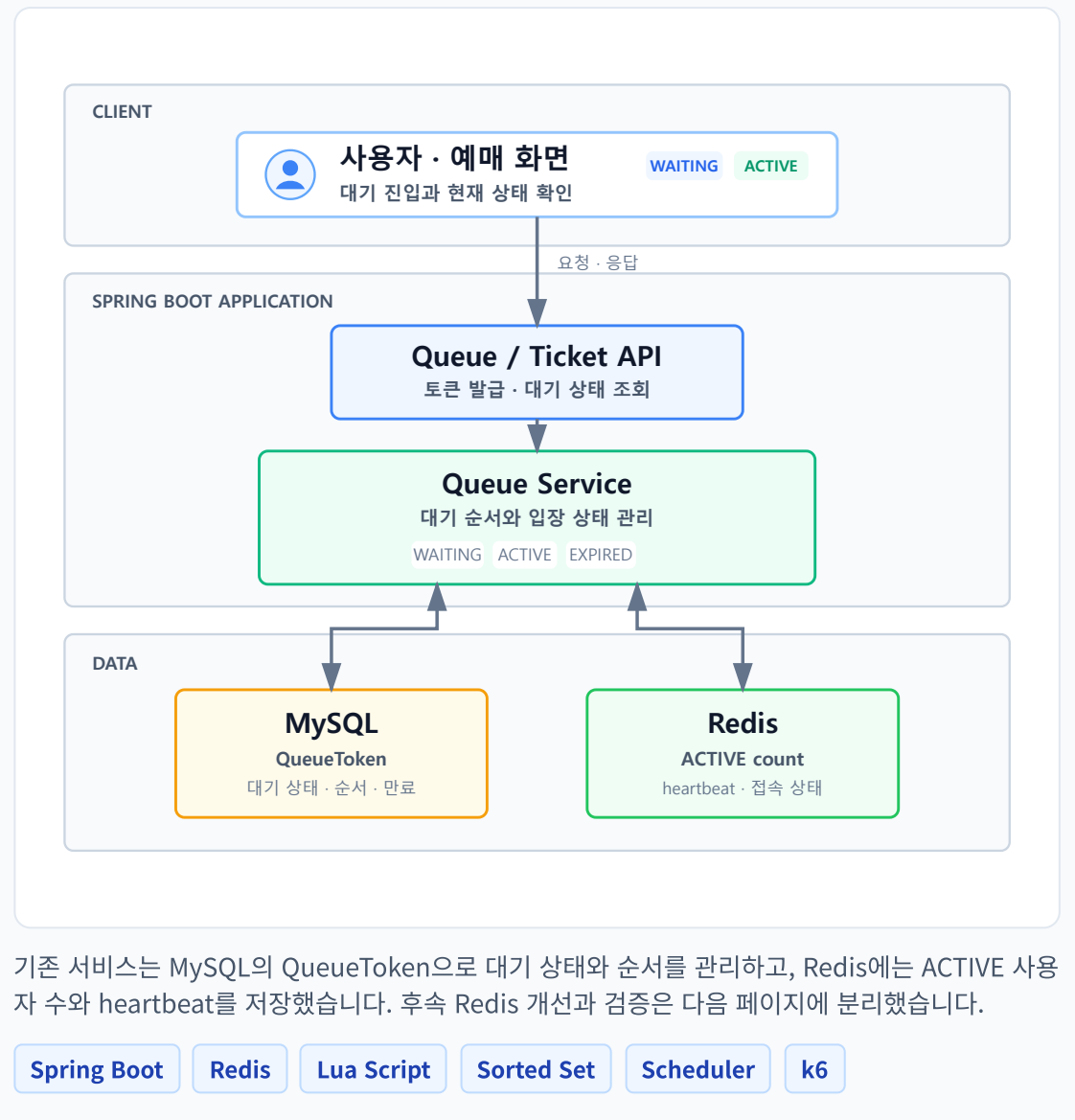
기간 2025.11 ~ 2025.12 (4주)

팀 구성 2명 (BE 1, FE 1)

GitHub github.com/dksehgd/Tget

시스템 아키텍처

System Architecture



프로젝트 종료 후 Redis 개선안 검증

01

발견·재검증

기존 대기열 수치만으로는 동시 요청의 ‘접수 순서’를 명확히 설명하기 어려웠습니다.

토른은 사용자를 식별하지만 요청 순서를 나타내지 않고, 시간값은 같은 시각에 들어온 요청의 서버 수신 순서를 구분하기 어렵다고 판단했습니다.

원인·영향

기존 구조에는 동시 요청의 접수 순서를 구분할 고유 순번이 없었습니다. 순번 발급·대기열 저장·상태 저장을 개별 명령으로 처리하면 일부만 반영될 위험도 있었습니다.

선택 이유·적용

INCR로 모든 서버가 공유하는 증가 순번을 만들고, 순위 조회와 이탈 제거가 가능한 Sorted Set에 저장했습니다. 세 명령은 Lua로 묶어 원자 처리했습니다.

검증 결과

100,000 / 100,000

순번 중복·누락 0건
저장 순서 100,000/100,000 일치
별도 비교 실험: 처리량 6.86% 증가

02

위험·검증

취소 요청이 없는 연결 종료에서도 서버가 슬롯을 회수할 기준이 필요했습니다.

브라우저 종료 시 취소 요청 전달을 보장할 수 없으므로, Heartbeat가 끊긴 뒤 서버가 슬롯을 회수해 다음 사용자를 입장시키는 시간을 검증 항목으로 정했습니다.

원인·영향

정상 취소는 즉시 처리되지만 연결 종료는 클라이언트의 취소 요청을 보장할 수 없어 서버가 이탈 여부를 판단해야 했습니다.

선택 이유·적용

서버가 마지막 응답 시각을 기준으로 회수하도록 Heartbeat와 connection lease를 두고 scheduler가 만료 슬롯을 정리하게 했습니다.

검증 결과

15.107초

ACTIVE 100명 통신 중단 후
101번 사용자 ACTIVE 전환

테스트 설정 15초 lease·1초 scheduler는 회수 동작을 확인하기 위한 값이며 운영 최적값은 아닙니다.

검증 범위 loadtest 전용 Spring HTTP 경로만 사용했고 인증·MySQL·좌석·예약·결제는 제외했습니다. 10만 건 순서 검증에서는 scheduler를 비활성화했으며, 실제 /queue 경로와 Redis·다중 서버 장애 복구는 후속 과제입니다.

기업 연계 · LLM 크롤링 · 결과 검증

S-Catch

삼성닷컴 글로벌 상품 정보를 수집하고 기존 데이터와 비교한 기업 연계 프로젝트.

Playwright 브라우저 제어·DOM 추출, OpenAI API 연동, 생성 코드 실행과 기존 데이터 비교 검증을 담당했습니다.

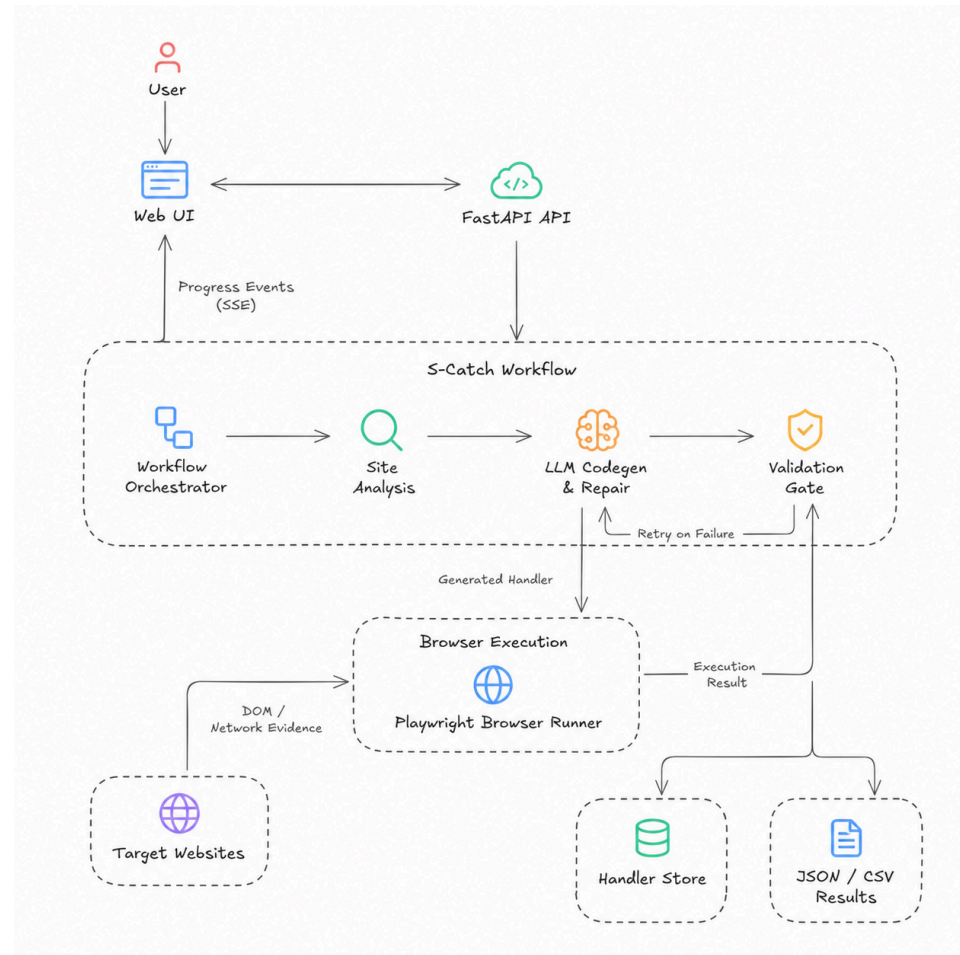
기간 2026.02 ~ 2026.04 (6주)

팀 구성 6명

GitHub github.com/dksehgd/s-catch

시스템 아키텍처

System Architecture



제한된 미팅에서 요구사항을 확인하기 위해 데모를 제안·구현했습니다. 요구사항 구체화와 오류 재시도, FastAPI 엔드포인트는 팀원과 공동 개발했습니다.

FastAPI

Playwright

OpenAI API

JSON

Result Validation

제한된 소통을 데모와 검증 흐름으로 구체화

01

상황·판단

현업자→멘토→팀으로 의견이 전달되고 직접 미팅이 적어, 말로 정리한 요구사항만으로는 해석 차이가 남았습니다.

수집 항목·결과물·실패 조건을 함께 정리한 뒤, 제한된 미팅에서 상대가 직접 확인하고 의견을 줄 수 있는 형태가 필요하다고 판단했습니다.

소통의 제약

질문을 여러 차례 주고받기 어려워 한 번의 미팅에서 해석 차이를 최대한 드러내야 했습니다.

대안·실행

미팅용 데모를 먼저 제안하고 화면과 기능을 구현해, 제가 맡은 수집·검증 흐름을 직접 시연하고 현업 의견을 구현 방향에 반영했습니다.

결과

우수상

기업 연계 프로젝트 최종 우수상

02

발견·판단

국가별 삼성닷컴은 DOM과 옵션 구조가 달랐고, 일부 페이지는 HTML과 API 응답을 함께 사용했습니다.

고정된 수집 규칙만으로는 변경 대응이 어려웠고, LLM 생성 코드를 그대로 채택하면 실행에 성공해도 잘못 수집된 값을 놓칠 수 있어 생성과 검증을 분리했습니다.

기술 선택 이유

동적 화면을 실제 브라우저처럼 조작하고 DOM과 네트워크 응답을 함께 확인하기 위해 Playwright를 사용했습니다.

적용

추출 정보를 OpenAI API에 전달해 페이지별 코드를 생성·실행하고, 실행 오류와 비교 결과를 팀원과 공동 구현한 재시도 흐름에 전달했습니다.

개인 구현 범위

브라우저 접근·DOM 추출·API 호출·생성 코드 실행·기준 데이터 비교를 담당했습니다. 오류 재시도와 FastAPI 엔드포인트는 공동 구현했습니다.

검증 범위 생성 코드의 실행 여부와 기준 데이터 비교 절차까지 구현했습니다. 운영 배포와 페이지 변경에 대한 장기 회귀 검증은 후속 과제입니다.

THANK YOU

로그와 지표로 문제를 확인하고, 검증 가능한 구조로 개선하는 개발자 안도형이었습니다.

